Python – Concepts de bases

Python est un langage de programmation polyvalent open source, conçu pour être simple à lire et à écrire. Il est utilisé dans de nombreux domaines : développement web, analyse de données, automatisation, enseignement, IA, etc. Sa syntaxe claire et intuitive en fait un excellent choix pour les débutants comme pour les professionnels. Il est très apprécié pour sa productivité et sa lisibilité, et une communauté très nombreuse peut être source d'aide.

Les variables

Affecter une valeur à une variable

Les variables sont des boîtes, avec une étiquette (leur nom), et qui contiennent une donnée. On les nomme en séparant les mots par un "_" (snake_case).

Dans les lignes ci-dessous, les textes en clair sont des commentaires.

- Pour un nombre entier :

```
ma_variable_1 = 3  # la variable a pour valeur le nombre entier 3
```

- Pour un nombre décimal :

```
ma_variable_2 = 3.14  # la variable a pour valeur le nombre 3.14
```

- Pour une chaîne de caractères :

```
ma_variable_3 = "mon prénom"  # la variable a pour valeur le texte "mon prénom"
```

- Pour un booléen :

```
ma_variable_4 = True# la variable a pour valeur True
```

Appeler et afficher une variable

On utilise une variable créée plus haut.

- On peut faire des calculs :

```
somme = ma_variable_1 + ma_variable_2  # calcul d'une somme, stockage dans une autre variable
```

- On peut afficher un résultat :

```
print(somme)  # on affiche le résultat brut
```

- On peut utiliser des chaînes de caractères formatées, ou `f-string`. Une `f-string` est une chaîne de caractères qui comporte une variable.

```
montant = 250
print(f"Votre compte est crédité de {montant} €")
```

Principaux types de variables

Les variables ont un type donné. Chaque type est utilisable d'une façon qui lui est propre.

- Pour connaître le type d'une variable, on peut taper la commande :

```
print(type(ma_variable))  # on affiche le type de la variable
```

- Un nombre entier est de type `int`.
- Un nombre décimal est de type `float`.
- Une chaîne de caractères est de type `str`.
- Un booléen est de type `bool`. Il peut prendre la valeur `True` ou `False`.

Erreurs fréquentes

- Oubli des `""` dans une chaîne de caractères : `SyntaxError`.
- Oubli des `()` de `print()` : `SyntaxError`.
- Oubli de `f` ou des `{}` dans une `f-string` : `comportement anormal`.

Exercices

Lire du code : qu'affiche le code suivant ?

```
print("Hello")
print("Python")
```

- message d'erreur
- `Hello Python`
- `Hello`
  `Python`
- `Hello`

Lire et écrire du code : corriger les erreurs du code suivant.

```
print(Bienvenue en Python")
print Bien joué !
```

Écrire du code : coder le programme suivant.

- Créer une variable contenant votre prénom, et une autre contenant votre âge.
- Afficher un message du type : « `Bonjour Paul, vous avez 18 ans.` » en utilisant les `f-string`.

Manipulation de variables

Calculer en Python

Python est un excellent calculateur. Il réalise de nombreuses opérations avec les symboles suivants (les priorités sont les mêmes qu'en mathématiques) :

addition	`+`	`5 + 2`	`7`	division (quotient entier)	`//`	`5 // 2`	`2`
soustraction	`-`	`5 - 2`	`3`	division (reste)	`%`	`5 % 2`	`1`
multiplication	`*`	`5 * 2`	`10`	puissance	`**`	`5 ** 2`	`25`
division	`/`	`5 / 2`	`2.5`				

```
print(5 + 2)  # Python affiche 7
```

Récupérer une saisie utilisateur

On peut récupérer des données pour établir un "dialogue" entre l'ordinateur et l'utilisateur. Attention, la variable sera toujours de type `str`.

```
age = input("Saisissez votre âge : ")  # le texte permet de guider l'utilisateur
```

Changer le type d'une variable

La fonction `input()` permet de récupérer une chaîne de caractères. Mais on peut avoir besoin de la convertir en nombre.

```
ma_variable_1 = int("32")  # renvoie le nombre entier 32
ma_variable_2 = float("37.5")  # renvoie le nombre décimal 37.5
ma_variable_3 = bool(1)  # renvoie le booléen True (pour 0, renvoie False)
ma_variable_4 = str(32)  # renvoie la chaîne de caractères "32"
ma_variable_5 = str(10.23)  # renvoie la chaîne de caractères "10.23"
ma_variable_6 = str(False)  # renvoie la chaîne de caractères "False"
```

Erreurs fréquentes

- Calcul avec des variables de types différents (ex : `"Coucou" + 3`) : `TypeError`.
- Conversion d'une chaîne de caractères en nombre (ex : `int("abc")`) : `ValueError`.
- Oubli du fait que `input()` renvoie une chaîne de caractères : `TypeError`.

Exercices

Lire du code : qu'affiche le code suivant ?

```
a = 10
b = 2
c = a * b
print(c)
```

Lire et écrire du code : corriger les erreurs du code suivant.

```
note_1 = 14
note_2 = 12
note_3 = input("Note : ")

moyenne = (note_1 + note_2 + note_3) / 3
print(f"La moyenne est de {moyenne}")
```

Écrire du code : coder le programme suivant.

- Demander à l'utilisateur de saisir 2 nombres entiers.
- Les stocker dans des variables.
- Les convertir en entiers.
- Calculer la somme de ces deux nombres et stocker le résultat dans une variable.
- Afficher le résultat avec le calcul, par exemple : « `5 + 3 = 8` ».

Les structures conditionnelles

Tester une condition

Un test de condition renvoie un booléen (`True` ou `False`).

- Cas n° 1 : "si..."

```
if ma_variable == 0:  # si cette condition est vraie
    print("la variable est nulle")  # on affiche ce texte
```

- Cas n° 2 : "si... sinon..."

```
if ma_variable > 0:  # si cette condition est vraie
    print("la variable est positive")  # on affiche ce texte
else:  # sinon
    print("la variable est négative ou nulle")  # on affiche ce texte
```

- Cas n°3 : "si... sinon si... sinon"

```
if ma_variable < 0:  # si cette condition est vraie
    print("la variable est négative")  # on affiche ce texte
elif ma_variable > 0:  # sinon si cette autre condition est vraie
    print("la variable est positive")  # on affiche ce texte
else:  # sinon
    print("la variable est nulle")  # on affiche ce texte
```

Opérateurs de comparaison

`==`	... est égal à ...	`!=`	... est différent de ...	`<`	... est strictement inférieur à ...
`>`	... est strictement supérieur à ...	`<=`	... est inférieur ou égal à ...	`>=`	... est supérieur ou égal à ...

Exemple :

```
if ma_valeur >= 0:  # si la valeur de la variable est positive ou nulle
```

Tester plusieurs conditions : les opérateurs logiques

`and`	... et ... (combine deux tests, les deux doivent être vrais)	`or`	... ou ... (combine deux tests, un seul doit être vrai)	`not`	... non ... (inverse un test)

Ces opérateurs permettent de combiner deux comparaisons.

- Utilisation de `and` :

```
if ma_valeur_1 >= 0 and ma_valeur_2 < 0:  # les deux conditions doivent être respectées
```

- Utilisation de `or` :

```
if ma_valeur_3 == 0 or ma_valeur_4 < 0:  # une seule des deux conditions doit être
respectée
```

- Utilisation de `not` :

```
if utilisateur_connecte == False:  # syntaxe standard ; correcte mais peu intuitive

if not utilisateur_connecte:  # syntaxe équivalente avec not, plus intuitive
```

Chaîner des comparaisons

```
if ma_valeur >= 0 and ma_valeur < 50:  # syntaxe standard ; correcte mais peu intuitive

if 0 <= ma_valeur < 50:  # syntaxe équivalente avec chaînage, plus facile à lire
```

Erreurs fréquentes

- Confusion de `=` avec `==` : `SyntaxError`.
- Oubli des `:` après `if`, `elif` ou `else` : `SyntaxError`.
- Oubli des indentations après `if`, `elif` ou `else` : `IndentationError`.
- Condition impossible à atteindre : `comportement anormal`.

Exercices

Lire du code : qu'affiche le code suivant ?

```
x = 12
if x > 10:
    print("grand")
elif x < 20:
    print("moyen")
else:
    print("petit")
```

Lire et écrire du code : corriger les erreurs du code suivant.

```
if temperature <= 0
print("Il gèle")
else:
print("Il ne gèle pas")
```

Écrire du code : coder le programme suivant.

- Récupérer l'âge de l'utilisateur.
- Selon l'âge saisi, afficher le message « `Vous êtes majeur` » ou « `Vous êtes mineur` ».
- Demander à l'utilisateur de renseigner son sexe.
- Modifier votre code pour que les mots « `majeur` » et « `mineur` » s'accordent.

Les boucles

Créer une boucle `for`

Cette boucle permet de répéter une série d'opérations un nombre déterminé de fois.

```
for i in range(10):  # pour chaque nombre entre 0 et 9
    print(i * 2)  # on affiche le double du nombre
    print(i ** 2)  # on affiche le carré du nombre
```

Créer une boucle `while`

Cette boucle permet de répéter une série d'opérations tant qu'une condition est vérifiée.

```
saisie_utilisateur = ""  # on crée une variable vide
while saisie_utilisateur != "oui":  # tant que la variable n'est pas égale à "oui"
    saisie_utilisateur = input("Tapez 'oui' en minuscules : ")  # on demande à
l'utilisateur de taper "oui"
```

Sortir d'une boucle `while`

Dans certains cas, une boucle `while` peut être infinie. Pour en sortir, on utilise le mot `break`.

```
while True:  # boucle infinie : True est toujours vrai...
    lettre = input("Taper une lettre : ")  # on demande une saisie à l'utilisateur
    if lettre == "a":  # on donne une condition
        print("OK !")  # on affiche un message de validation
        break  # on sort de la boucle si la condition est vraie
    else:  # sinon...
        print("mauvais choix")  # on affiche un message d'erreur
```

Erreurs fréquentes

- Oubli des `:` à la fin de la ligne `for` ou `while` : `SyntaxError`.
- Indentation incorrecte : `IndentationError`.
- Boucle infinie involontaire (oubli de `break`) : `le programme semble bloqué`.
- Mauvaise utilisation de `range()` : `comportement anormal`.

Exercices

Lire du code : qu'affiche le code suivant ?

Boucle `while`

```
password = ""
i = 0
while password != "1234":
    password = input("Mot de passe : ")
    i = i + 1
```

Boucle `for`

```
for i in range(3):
    print("Python")
```

Lire et écrire du code : corriger les erreurs du code suivant.

Boucle `while`

```
i = 0
while i < 5
print(i)
i = i + 1
```

Boucle `for`

```
for i in range(5)
print(i)
print("Fin du programme")
```

Écrire du code : coder le programme suivant.

Boucle `while`

- Demander à l'utilisateur de taper « `ok` » jusqu'à ce qu'il le fasse vraiment.

Boucle `for`

- Demander à l'utilisateur de saisir un nombre entier.
- Afficher la table de multiplication complète de ce nombre.

Projet : la calculatrice

Énoncé

Réaliser une calculatrice en Python. Votre programme devra respecter le cahier des charges.

FORMAT DES DONNÉES :

- Les données sont stockées dans des variables.

FONCTIONNEMENT :

- L'utilisateur saisit un nombre, puis un signe opératoire, puis un autre nombre.
- L'ordinateur affiche le calcul et son résultat.
- L'utilisateur peut décider de faire autant de calculs qu'il le veut avant de quitter.

Conseils de réalisation

- Stocker les saisies utilisateur dans des variables aux noms explicites.
- Penser à convertir les nombres en `float`.
- Coder une seule opération dans un premier temps (l'addition).
- Utiliser une `f-string` pour afficher le résultat proprement.
- Envisager les 4 possibilités d'opérations avec une structure conditionnelle.
- Ajouter une boucle pour recommencer.

Python – Concepts intermédiaires

Les listes

Créer une liste

- On peut créer une liste vide pour la compléter ensuite ; on utilise les `[]`.

```
liste_vide = []
```

- On peut aussi créer une liste qui contient déjà des éléments :

```
ingredients = ["farine", "sucre", "lait"]
```

Ajouter un élément dans une liste

Le nouvel élément est toujours ajouté à la fin de la liste.

```
ingredients.append("oeufs")
```

Trouver un élément dans une liste

- Pour récupérer la position d'un élément dans la liste (son index) :

```
ingredients = ["farine", "sucre", "lait", "oeufs"]
numero = ingredients.index("sucre")  # les index commencent à 0, donc le "sucre" est à l'index 1 : les index sont 0 ; 1 ; 2 ; 3. Ici, numero prendra la valeur 1
```

- Pour récupérer le 3ème élément dans la liste :

```
element = ingredients[2]  # les index commencent à 0. Ici, element prendra la valeur "lait"
```

Supprimer un élément d'une liste

- Ce code efface complètement l'élément supprimé.

```
ingredients = ["farine", "sucre", "lait", "oeufs"]
ingredients.remove("lait")
```

- Ce code permet de stocker l'élément supprimé dans une variable.

```
ingredients = ["farine", "sucre", "oeufs"]
element = ingredients.pop(1)  # dans notre cas, element sera égal à "sucre"
```

Connaître la longueur d'une liste

Cela peut être utile de connaître le nombre d'éléments dans une liste.

```
ingredients = ["farine", "sucre", "oeufs"]
len(ingredients)  # renvoie 3
```

Trier une liste

Dans une liste de nombres, le tri est croissant. Dans une liste de chaînes, le tri est alphabétique.

```
ingredients.sort()
```

Parcourir une liste

On réalise une boucle sur la liste, et chaque élément de la liste sera traité indépendamment.

```
for element in ingredients:  # pour chaque élément de la liste "ingredients"
    print(element)  # affiche l'élément
```

Erreurs fréquentes

- Oubli des `[]` dans une liste : `SyntaxError`.
- Oubli des `,` de séparation : `SyntaxError` ou `comportement anormal`.
- Utilisation d'un index trop grand : `IndexError`.

Exercices

Lire du code : qu'affiche le code suivant ?

```
n = [3, 6, 9, 12]
x = n.pop(1)
print(x)
print(n)
```

Lire et écrire du code : corriger les erreurs du code suivant.

```
l = [4, 7, 1]
l.sort
print(l(0))
```

Écrire du code : coder le programme suivant.

- Créer une liste de toutes les matières enseignées au collège.
- Afficher la liste.
- Ajouter les options proposées au collège à la liste.
- Afficher la liste.
- Trier les matières par ordre alphabétique.
- Afficher la liste.
- Enlever les matières que vous ne pratiquez pas.
- Afficher le nombre de matières que vous pratiquez.
- Afficher les matières une par une.

Les tuples

Créer un tuple

Un tuple fonctionne comme une liste, mais une fois créé, il n'est pas modifiable. On peut l'écraser, mais pas modifier les valeurs qu'il contient. On utilise les `()`.

```
coordonnees = (45, 32)
```

Trouver un élément dans un tuple

- Pour récupérer la position d'un élément dans le tuple (son index) :

```
semaine = ("lun", "mar", "mer", "jeu", "ven", "sam", "dim")
numero = semaine.index("jeu")  # les index commencent à 0, donc le jeudi est à l'index
3 : les index sont 0 ; 1 ; ... 6. Ici, numero prendra la valeur 3
```

- Pour récupérer le 6ème élément dans le tuple :

```
jour = semaine[5]  # les index commencent à 0. Ici, jour prendra la valeur "sam"
```

Connaître la longueur d'un tuple

```
semaine = ("lun", "mar", "mer", "jeu", "ven", "sam", "dim")
len(semaine)  # renvoie 7
```

Parcourir un tuple

On réalise une boucle sur le tuple, et chaque élément du tuple sera traité indépendamment.

```
for jour in semaine:  # pour chaque élément du tuple "semaine"
    print(jour)  # affiche la valeur
```

Tuple ou liste ?

Les tuples sont moins maniables (non modifiables), mais ils nécessitent moins de mémoire que les listes. Si vous voulez protéger les données en écriture, utilisez un tuple. Si vous voulez les rendre modifiables, utilisez une liste.

Erreurs fréquentes

- Tentative de modification d'un tuple : `AttributeError`.
- Oubli des `,` de séparation : `SyntaxError` ou `comportement anormal`.

Exercices

Lire du code : qu'affiche le code suivant ?

```
a, b = (3, 7)
print(a + b)
```

Lire et écrire du code : corriger les erreurs du code suivant.

```
mois = ("J" "F" "M" "A" "M" "J" "J" "A" "S" "O" "N" "D")
mois.sort()
for mois in mois:
    print(mois)
```

Écrire du code : coder le programme suivant.

- Créer un tuple avec votre prénom, votre nom et votre date de naissance.
- Afficher une phrase du type : « `Bastille Bastien - né(e) le 14/07/1789` ».

Manipulation de chaînes de caractères

Changer la casse d'une chaîne de caractères

Changer la casse, c'est passer des majuscules aux minuscules.

```
variable = "cOuCoU mAcHiN"
variable.lower()  # tout en minuscules : "coucou machin"
variable.upper()  # tout en majuscules : "COUCOU MACHIN"
variable.capitalize()  # majuscule à la première lettre du premier mot : "Coucou
machin"
variable.title()  # majuscule à la première lettre de chaque mot : "Coucou Machin"
```

Connaître la longueur d'une chaîne de caractères

Cela peut être utile de connaître la longueur d'une chaîne de caractères.

```
variable = "Une chaîne de caractères"
len(variable)  # renvoie 24
```

Erreurs fréquentes

- Oubli des parenthèses après l'appel du changement de casse : `comportement anormal`.
- Mauvaise utilisation de `len()` ou des méthodes d'appel de changement de casse : `AttributeError` ou `NameError`.

Exercices

Lire du code : qu'affiche le code suivant ?

```
s = "bonjour"
print(s.capitalize())
```

Lire et écrire du code : corriger les erreurs du code suivant.

```
titre = "le petit prince"
titre = title(titre)
print(titre)
print(titre.len())
```

Écrire du code : coder le programme suivant.

- Demander à l'utilisateur de saisir son nom et son prénom.
- Enregistrer le nom tout en majuscules, le prénom avec la première lettre en majuscules.

Les fonctions

Définir une fonction simple

Les fonctions ont pour but de réaliser une série de tâches. On les nomme en séparant les mots par "_" snake_case.

```
def dire_bonjour():  # initialisation de la fonction
    print("Bonjour !")
```

Définir une fonction avec des paramètres

La fonction réalise une série de tâches, mais qui dépendent de paramètres qu'on lui donne.

```
def dire_bonjour_nom(nom):  # nom est un paramètre à donner à la fonction
    print(f"Bonjour, {nom} !")
```

Récupérer le résultat d'une fonction

Quand on veut récupérer le résultat d'une fonction dans une variable, on utilise le mot `return`.

```
def test_temperature(valeur):
    if valeur <= 0:
        message = "il gèle !"
    else:
        message = "il ne gèle pas."
    return message  # la fonction renvoie une valeur grâce à return
```

Appeler une fonction

Nous avons déjà appelé des fonctions : `input()`, `print()`, `type()`... sont des fonctions. Les trois fonctions ci-dessus sont appelées successivement :

```
dire_bonjour()

dire_bonjour_nom("Toto")

reponse = test_temperature(14.5)
print(reponse)
```

Erreurs fréquentes

- Oubli des `()` en appelant la fonction : `SyntaxError`.
- Mauvaise gestion des indentations : `IndentationError`.
- Mauvaise gestion des paramètres donnés : `TypeError`.
- Oubli de `return` : `comportement anormal`.
- Confusion `print`/`return` dans une fonction : `comportement anormal`.

Exercices

Lire du code : qu'affiche le code suivant ?

```
def calc(x):
    return x + 3

print(calc(7))
```

Lire et écrire du code : corriger les erreurs du code suivant.

```
def triple x:
return x*3

y = triple(4
print(y)
```

Écrire du code : réaliser le code suivant.

- Créer une fonction qui renvoie le carré et le cube d'un nombre (sous forme d'un tuple).

Les dictionnaires

Créer un dictionnaire

- On peut créer un dictionnaire vide pour le compléter ensuite ; on utilise les `{}`.

```
dictionnaire_vide = {}
```

- On peut aussi créer un dictionnaire qui contient déjà des éléments :

```
eleve = {"nom": "Toto", "note": 14.5}  # clés : "nom" et "note", valeurs : "Toto" et
14.5
```

Ajouter ou modifier un élément dans un dictionnaire

La syntaxe est particulière : on donne la clé dans des `[]`, et la valeur après le `=`. C'est la même syntaxe pour modifier une valeur dont la clé existe déjà : on écrase les données.

```
eleve["age"] = 12  # ici, l'âge est ajouté au dictionnaire
eleve["note"] = 15  # ici, la valeur de la note est remplacée
```

Trouver un élément dans un dictionnaire

On cherche la valeur en fonction de la clé.

```
print(eleve["nom"])  # dans ce cas, on affichera "Toto"
```

Supprimer un élément d'un dictionnaire

On utilise le mot `del`.

```
personnage = {"nom": "Dark Vador", "categorie": "Boss", "force": 18, "arme": "sabre
laser"}
del personnage["arme"]
```

Parcourir un dictionnaire

Il y a plusieurs manières de parcourir un dictionnaire avec une boucle `for`.

```
for cle in dictionnaire.keys():  # parcourt le dictionnaire selon les clés
for valeur in dictionnaire.values():  # parcourt le dictionnaire selon les valeurs
for cle, valeur in dictionnaire.items():  # parcourt le dictionnaire selon les deux
```

Erreurs fréquentes

- Appel d'une clé inexistante : `KeyError`.
- Confusion liste/dictionnaire : `KeyError` ou `comportement anormal`.

Exercices

Lire du code : qu'affiche le code suivant ?

```
d = {"a": 3, "b": 7}
d["a"] = d["a"] + 10
print(d)
```

Lire et écrire du code : corriger les erreurs du code suivant.

```
notes = {math: 15, "anglais": 12}
print(notes[math])
```

Écrire du code : réaliser le code suivant.

- Créer un dictionnaire correspondant à un livre, avec son titre, son auteur et l'année d'édition.
- Afficher chaque donnée séparément.

Projet : Le fichier clients

Énoncé

Une entreprise vous sollicite pour créer une application qui permettra de gérer ses clients. Vous devez respecter le cahier des charges.

FORMAT DES DONNÉES :

- Les clients sont stockés dans une liste.
- Chaque client est enregistré sous la forme d'un dictionnaire avec son nom, son prénom, son âge.
- Le nom doit être en majuscules, le prénom avec seulement les premières lettres en majuscules et l'âge doit être un nombre entier.

FONCTIONNEMENT :

- Lorsque l'utilisateur lance le programme, il accède à un menu. Le menu lui permet de :
  - saisir un nouveau client.
  - supprimer un client.
  - afficher la liste des clients.
  - afficher le nombre de clients.
- L'application fonctionne tant que l'utilisateur n'a pas demandé à en sortir.

Conseils de réalisation

- Créer une liste vide `clients`.
- Coder la fonction `ajouter_client()`.
- Coder les fonctions `afficher_clients()` et `compter_clients()`.
- Coder la fonction `supprimer_client()`.
- Dans les fonctions `ajouter_client()` et `supprimer_client()`, il y a une partie commune. Créer la fonction `saisir_client()` et l'appeler dans `ajouter_client()` et `supprimer_client()`.
- Ajouter la mise en forme des données clients en manipulant les chaînes de caractères.
- Ajouter un menu, puis une boucle.

Python – Concepts avancés

Les classes

Créer une classe

Imaginons la classe `Personnage`. Chaque personnage a un nom, un âge, ... : ce sont ses attributs. Il peut aussi faire des choses : parler, ... : ce sont ses méthodes. Créer la classe `Personnage` équivaut à concevoir un plan pour créer n'importe quel personnage. Tous les personnages ainsi créés auront les mêmes attributs, avec des valeurs différentes, et les mêmes méthodes. On nomme une classe en mettant une majuscule à chaque mot (PascalCase).

```
class Personnage:
    def __init__(self, nom, age):  # La méthode constructeur ; il faut lui donner 2
paramètres
        self.nom = nom  # l'attribut nom ; on lui affecte la valeur du paramètre nom
        self.age = age  # l'attribut age ; on lui affecte la valeur du paramètre age

    def parler(self):  # la méthode parler()
        print(f"Bonjour, je m'appelle {self.nom}.")
```

Créer une instance à partir d'une classe

On peut créer un grand nombre d'objets à partir de la classe : on parle d'instances de la classe. Ici, `perso_1` et `perso_2` sont deux instances de la classe `Personnage`. Pour créer une instance, il faut passer une valeur par paramètre de la méthode `__init__`.

```
perso_1 = Personnage("Luke", 25)  # on passe des valeurs aux paramètres du constructeur
perso_2 = Personnage("Yoda", 163)  # on crée une deuxième instance
```

Utiliser les attributs d'une instance

Chaque instance est accessible via son nom de variable (ici, `perso_1` et `perso_2`).

```
print(perso_1.nom)  # affiche "Luke"
print(perso_2.nom)  # affiche "Yoda"
print(perso_1.age)  # affiche "25"
print(perso_2.age)  # affiche "163"
```

Utiliser les méthodes d'une instance

On peut considérer que les méthodes sont des fonctions, mais uniquement valables pour les instances de la classe. Toutes les méthodes prennent `self` comme premier paramètre. Dans le code, il n'est pas nécessaire de le fournir lors de l'appel d'une méthode.

```
perso_1.parler()  # affiche "Bonjour ; je m'appelle Luke"
perso_2.parler()  # affiche "Bonjour ; je m'appelle Yoda"
```

La méthode spéciale constructeur : `__init__`

Cette méthode est appelée au moment de l'initialisation de l'instance. Il faut lui apporter les paramètres nécessaires pour qu'elle puisse affecter les valeurs voulues aux attributs de l'instance.

La méthode spéciale d'optimisation de lisibilité : `__str__`

- Par défaut, un `print()` d'une instance affiche quelque chose de peu lisible.

```
print(perso_1)  # affiche <__main__.Personnage object at 0x00000211175B6F90>
```

- Pour éviter cela, on crée la méthode `__str__`.

```
class Personnage:
    def __init__(self, nom, age):
        ...

    def __str__(self):
        return f"{self.nom}, {self.age} ans."
```

- Ensuite, on peut refaire un `print()`.

```
print(perso_1)  # affiche "Luke, 25 ans."
```

Erreurs fréquentes

- Oublier self : `TypeError`.
- Oublier les `()` avec une méthode : `comportement anormal`.
- Confondre un attribut d'instance avec une variable « standard » : `comportement anormal`.
- Mal orthographier `__init__` ou `__str__` : `TypeError` ou `comportement anormal`.

Exercices

Lire du code : qu'affiche le code suivant ?

```
class Voiture:
    def __init__(self, marque, modele, annee):
        self.marque = marque
        self.modele = modele
        self.annee = annee

    def __str__(self):
        return f"{self.marque} {self.modele} - {self.annee}"

    def demarrer(self):
        print(f"{self.marque} {self.modele} démarre.")

v1 = Voiture("Renault", "Twingo", 2023)
print(v1)
v1.demarrer()
```

Lire et écrire du code : corriger les erreurs du code suivant.

```
class Robot:
    def init(self, nom):
        self.nom = nom

    def parler()
        print(f"Bonjour, je suis {nom}")

r = Robot("R2-D2")
r.parler
```

Écrire du code : réaliser le code suivant.

- Créer une classe Rectangle avec 2 attributs (longueur, largeur) et 2 méthodes (aire, périmètre).
- Créer deux instances, une de dimensions 3 x 8 et l'autre de dimensions 5 x 6.
- Afficher le périmètre et l'aire des deux instances.

La gestion des erreurs

Principe général

Lorsqu'un programme rencontre une erreur, Python s'arrête immédiatement. Ce comportement est à éviter : il faut faire en sorte que le programme puisse continuer. C'est le rôle des blocs `try`/`except`.

```
try:  # Python essaie de suivre ces instructions
    x = int(input("Nombre : "))
    print(10 / x)
except:  # S'il rencontre des erreurs dans le bloc try, il suivra ces instructions
    print("Il y a une erreur.")
```

Types d'erreurs fréquents

`ValueError`	Conversion impossible	`int("abc")`
`ZeroDivisionError`	Division par 0	`37 / 0`
`TypeError`	Mélange de types de variables incompatibles	`"abc" + 3`
`KeyError`	Clé inexistante dans un dictionnaire	`moyenne["japonais"]`
`IndexError`	Index trop grand dans une liste ou un tuple	`matieres[32]`

Gérer plusieurs types d'erreurs

Dans le bloc `try`/`except` ci-dessus, il y a plusieurs causes d'erreur possible :

- L'utilisateur ne saisit pas un nombre (`ValueError`).
- L'utilisateur saisit 0 (`ZeroDivisionError`).

Il est conseillé de gérer chaque type d'erreur indépendamment. On peut donc améliorer le code :

```
try:  # Python essaie de suivre ces instructions
    x = int(input("Nombre : "))
    print(10 / x)
except ValueError:  # En cas de conversion impossible, il suivra ces instructions
    print("Vous devez saisir un nombre.")
except ZeroDivisionError:  # En cas d'erreur de division, il suivra ces instructions
    print("Il est impossible de diviser par 0.")
except:  # En cas d'erreur d'un autre type, il suivra ces instructions
   print("Erreur inconnue.")
```

Erreurs fréquentes

Mauvais ordre dans les `except` : `comportement anormal`.

Exercices

Lire du code : qu'affiche le code suivant ?

```
try:
    n = int("toto")
    print("OK")
except ValueError:
    print("Conversion impossible")
```

Lire et écrire du code : corriger les erreurs du code suivant.

```
try:
    print(10 / x)
except:
    print("Problème")
except ZeroDivisionError:
    print("Division impossible")
```

Écrire du code : réaliser le code suivant.

- Créer un programme qui :
  - demande à l'utilisateur un nombre entier (le numérateur).
  - demande un second nombre entier (le diviseur).
  - affiche le quotient de la division.
- Anticiper les erreurs de manipulation du programme.

Projet : Le parking

Énoncé

Une entreprise vous sollicite pour créer une application qui permettrait de gérer l'occupation de places de parking. Vous devez respecter le cahier des charges.

FORMAT DES DONNÉES :

- La classe `Place` : chaque place de parking est caractérisée par son numéro et son statut (libre ou occupé) ; une place peut changer de statut.
- La classe `Voiture` : chaque voiture est caractérisée par une plaque d'immatriculation et la place qu'elle occupe ; elle peut se garer ou partir.
- Les places de parking seront stockées dans un tuple.
- Les voitures stationnées seront stockées dans une liste.

FONCTIONNEMENT :

- Le parking comporte 5 places.
- Lorsque l'utilisateur lance le programme, il accède à un menu, qui permet de :
  - afficher le nombre de places libres.
  - afficher les plaques d'immatriculation des voitures garées.
  - faire entrer une voiture et la garer à une place libre aléatoire.
  - faire sortir une voiture.
- L'application gère les erreurs ou les anticipe avec des messages adaptés :
  - un véhicule cherche à occuper une deuxième place.
  - un véhicule non stationné cherche à sortir.
  - un véhicule cherche à stationner sur une place déjà occupée.
- L'application fonctionne tant que l'utilisateur n'a pas demandé à en sortir.

Conseils de réalisation

- Créer la classe `Place` avec les attributs `numero` et `occupee`, et la méthode `changer_occupation()`.
- Créer la classe `Voiture` avec les attributs `plaque` et `place`, et les méthodes `garer()` et `sortir()`.
- Créer les méthodes `__str__` de ces deux classes.
- Créer un parking de 5 places vides, numérotées de 1 à 5. Stocker ces données dans le tuple `parking`.
- Créer une liste `voitures`, vide pour l'instant.
- Créer la fonction `lister_places_libres()` qui renvoie la liste des places disponibles.
- Créer les fonctions associées au menu : `afficher_places_libres()`, `afficher_voitures()`, `garer_voiture()`, `sortir_voiture()`.
- Pour garer la voiture à une place libre aléatoire :
  - importer le module `random`.
  - déterminer la place avec `random.choice()`. Consulter la documentation sur le site de Python.
- Créer le menu (interface « agréable », claire et lisible).
- Boucler jusqu'à ce que l'utilisateur le décide.
- Gérer les erreurs de saisie utilisateur :
  - l'utilisateur saisit une plaque d'immatriculation vide.
  - l'utilisateur essaie de garer une voiture déjà existante dans le parking.
  - l'utilisateur essaie de garer une voiture sur un parking plein.
  - l'utilisateur essaie de sortir une voiture qui n'est pas sur le parking.